

Lab 3

Control structure and Data types in java

➤ Introduction to Scanner in Java

What is Scanner?

Scanner is a built-in Java class that allows us to read user input from the keyboard. We need to import it before using it.

How to Use Scanner?

To use Scanner, follow these steps:

1. **Import the Scanner class:** Add **import java.util.Scanner;** at the top of your program.
2. Create a Scanner object: **Scanner input = new Scanner(System.in);**

Explanation:

1. Scanner

Scanner is a built-in class in Java that is used to take input from the user.

2. Input

Input is just the name of the Scanner object.
You can name it anything, like scanner or userInput, but input is commonly used.

3. new Scanner(System.in)

new keyword is used to create a new object of the Scanner class.
Scanner(System.in) tells Java that we want to take input from the **keyboard** (console).
System.in is the standard input stream

3. Use methods to take input:

Example:

input.nextInt(); for numbers,
input.nextLine(); for text.

4. Close the Scanner: **input.close();**

Example of Scanner:

```

import java.util.Scanner; // Import Scanner class

public class ScannerExample {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in); // Create Scanner
        object

        // Asking for user input
        System.out.print("Enter your name: ");
        String name = input.nextLine(); // Read a string

        System.out.print("Enter your age: ");
        int age = input.nextInt(); // Read an integer

        System.out.println("Hello, " + name + "! You are " + age + "
        years old.");

        input.close(); // Close Scanner
    }
}

```

➤ **Scanner Methods:**

Method	Description
1) nextInt()	Reads an integer
2) nextDouble()	Reads a decimal number
3) nextLine()	Reads a full line of text
4) next()	Reads a single word
5) nextBoolean()	Reads true or false

➤ Understanding public static void main(String[] args) in Java

This is the **main method** in Java, and it is the entry point of every Java program. Java starts executing the program from this method. Now, let's explain each part:

1. public

public is an **access modifier** that means this method can be accessed from anywhere in the program.

Since Java programs start execution from main(), it must be **public** so that Java can access it.

2. static

The static keyword means that this method belongs to the **class** and not to an object.

This allows Java to call main() **without creating an object of the class**.

If main() were **not static**, we would have to create an object before calling it, which would be unnecessary.

3. void

void means **this method does not return any value**.

The main() method is just for execution, so it doesn't need to return anything.

4. main

This is the **name of the method**.

Java specifically looks for main() as the starting point of the program.

5. (String[] args)

String[] args is an **array of strings** that can store command-line arguments when running the program.

If no command-line arguments are given, args will be an empty array.

➤ IN short we can say that:

public: Accessible by Java runtime.

static: No object creation required.

void: No return value.

main: Starting point of execution.

(String[] args): Stores command-line arguments.

Control Structures in Java

1. If-Else Statement

The if-else statement helps to make decisions in a program.

Syntax:

```
if (condition) {  
    // Code runs if condition is true  
} else {  
    // Code runs if condition is false  
}
```

Example:

```
import java.util.Scanner;  
public class IfElseExample {  
    public static void main(String[] args) {  
        Scanner input = new Scanner(System.in);  
  
        System.out.print("Enter your age: ");  
        int age = input.nextInt();  
  
        if (age >= 18) {  
            System.out.println("You are an adult.");  
        } else {  
            System.out.println("You are a minor.");  
        }  
  
        input.close();  
    }  
}
```

2. Loops (For Loop & While Loop)

Loops help to run a block of code multiple times.

➤ For Loop

Used when the number of repetitions is known.

Syntax:

```
for (initialization; condition; update) {  
    // Code runs in loop  
}
```

Example:

```
public class ForLoopExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Number: " + i);  
        }  
    }  
}
```

➤ While Loop

Used when the number of repetitions is unknown.

Syntax:

```
while (condition) {  
    // Code runs in loop  
}
```

Example:

```
public class WhileLoopExample {  
    public static void main(String[] args) {  
        int number = 1;  
        while (number <= 5) {  
            System.out.println("Number: " + number);  
            number++;  
        }  
    }  
}
```

Data Types in Java

Java has different data types that store different kinds of values.

Java has **two main types** of data types:

1. **Primitive Data Types** (Basic built-in types)
2. **Non-Primitive Data Types** (Reference types like objects and arrays)

1. Primitive Data Types:

Primitive data types are the most basic types in Java, and they store simple values directly in memory.

Data Type	Size	Example
1. int	4 bytes	10
2. double	8 bytes	3.14
3. char	2 bytes	'A'
4. boolean	1 byte	true or false

2. Non-Primitive Data Types

Non-primitive data types are more complex and can store multiple values or methods. They do not store data directly in memory but instead reference objects.

Data Type	Description
String	A sequence of characters
Array	A collection of values of the same type
Class	A user-defined blueprint for objects
Interface	A set of abstract methods that a class must implement

➤ Difference Between Primitive and Non-Primitive Data Types in Java

1. Memory Storage

1. **Primitive data types** store actual values directly in memory.
2. **Non-primitive data types** store a reference (memory address) of the object.

2. Size

1. **Primitive types** have a fixed size (e.g., int is always 4 bytes).
2. **Non-primitive types** have variable size depending on the object.

3. Operations

1. **Primitive types** can be used directly with arithmetic operations (+, -, *, /).
2. **Non-primitive types** require methods to manipulate data (e.g., String.length()).

4. Complexity

1. **Primitive types** are simple and store only one value.
2. **Non-primitive types** can store multiple values and have methods.

5. Examples

1. **Primitive types:** int, char, boolean, float, double
2. **Non-primitive types:** String, Array, Class, Interface

6. Default Values

1. **Primitive types** have default values (e.g., int is 0, boolean is false).
2. **Non-primitive types** have null as the default value.

7. Usage

1. **Primitive types** are mainly used for calculations and simple data storage.
2. **Non-primitive types** are used for complex data structures and object-oriented programming.

➤ Some examples for Primitive data types

1. int (Integer)

```
import java.util.Scanner; // Import Scanner class

public class IntInputExample {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in); // Create Scanner
        object
        System.out.print("Enter an integer: ");
        int number = input.nextInt(); // Read integer input
        System.out.println("You entered: " + number);
    }
}
```

```
        input.close(); // Close scanner
    }
}
```

2. char (Character)

```
import java.util.Scanner;

public class CharInputExample {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a character: ");
        char letter = input.next().charAt(0); // Read first character of
input
        System.out.println("You entered: " + letter);
        input.close();
    }
}
```

3. boolean (True/False)

```
import java.util.Scanner;

public class BooleanInputExample {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Is Java fun? (true/false): ");
        boolean isJavaFun = input.nextBoolean(); // Read boolean
input
        System.out.println("You said: " + isJavaFun);
        input.close();
    }
}
```

4. Float

```
import java.util.Scanner;

public class FloatInputExample {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter a floating-point number: ");
        float pi = input.nextFloat(); // Read float input
        System.out.println("You entered: " + pi);
        input.close();
    }
}
```



```
}  
}
```

5. Double

```
import java.util.Scanner;  
  
public class DoubleInputExample {  
    public static void main(String[] args) {  
        Scanner input = new Scanner(System.in);  
        System.out.print("Enter a precise decimal number: ");  
        double preciseValue = input.nextDouble(); // Read double  
input  
        System.out.println("You entered: " + preciseValue);  
        input.close();  
    }  
}
```